

C# - Phần 10.8: LINQ Distinct()

Lý thuyết - Equality - Câu hỏi - Bài tập - Đáp án - Lưu ý

Mục tiêu: Hiểu Distinct() dùng để loại bỏ phần tử trùng, phân biệt loại trùng theo value/property với loại trùng object, theo dõi kiểu dữ liệu và biết kết hợp Select, Where, OrderBy, ToList.

PHẦN A - LÝ THUYẾT DISTINCT()

1. Distinct() dùng để làm gì?

Distinct() loại bỏ các phần tử trùng nhau và giữ lại một giá trị đại diện cho mỗi nhóm giá trị bằng nhau.

```
List<int> numbers = new List<int>
{
    1, 1, 2, 2, 3, 3
};

var result = numbers.Distinct();

1 1 2 2 3 3
  ↓ Distinct
1 2 3
```

2. Kiểu kết quả

```
var result = numbers.Distinct();
→ IEnumerable<int>

List<int> list =
    numbers
        .Distinct()
        .ToList();
→ List<int>
```

Distinct() trả về một collection/sequence, không trả về một phần tử duy nhất.

3. Distinct() không sửa collection gốc

Collection ban đầu vẫn giữ nguyên. Distinct() tạo một sequence kết quả mới.

Ghi nhớ: Distinct() loại trùng trong kết quả, không xóa trực tiếp phần tử khỏi List gốc.

4. Distinct() với string

```
List<string> names = new List<string>
{
    "Mouse",
    "Keyboard",
    "Mouse",
    "USB",
    "Keyboard"
};

var uniqueNames = names.Distinct();

Kết quả:
Mouse
Keyboard
USB
```

5. Phân biệt hoa thường với string

```
"Mouse"
"mouse"
"MOUSE"
```

Trong phép so sánh mặc định, các giá trị này được xem là khác nhau về chữ hoa/thường, nên Distinct() có thể giữ cả ba.

```
names.Distinct(  
    StringComparer.OrdinalIgnoreCase);
```

Có thể dùng comparer để bỏ trùng không phân biệt hoa/thường, nhưng comparer chưa phải trọng tâm của bài này.

6. Select() + Distinct()

Đây là cách rất thường gặp khi muốn loại trùng theo một property.

```
var uniquePrices =  
    products  
        .Select(  
            product => product.Price  
        )  
        .Distinct();  
  
List<Product>  
    ↓ Select Price  
IEnumerable<decimal>  
    ↓ Distinct  
IEnumerable<decimal>
```

Distinct() không đổi kiểu phần tử; nó chỉ loại các giá trị bị trùng.

7. Ví dụ Price bị trùng

```
Price:  
300  
700  
2500  
15000  
150  
300  
700  
  
Sau Distinct():  
300  
700  
2500  
15000  
150
```

8. Select(property).Distinct() khác products.Distinct()

```
products  
    .Select(p => p.Price)  
    .Distinct();
```

Ý nghĩa: kiểm tra sự trùng nhau của Price.

```
products.Distinct();
```

Ý nghĩa: kiểm tra sự trùng nhau của chính Product object.

Điểm cốt lõi: Trước khi dùng Distinct(), hãy tự hỏi: 'Trùng theo cái gì?'

PHẦN B - DISTINCT() VỚI OBJECT VÀ EQUALITY

9. Vì sao object phức tạp hơn int/decimal/string?

Với int, decimal, string, .NET đã có quy tắc equality phù hợp để so sánh các giá trị.

```
300 và 300  
→ bằng nhau
```

```
"Mouse" và "Mouse"  
→ bằng nhau
```

Với class Product do ta tự tạo, cần biết hai Product được xem là bằng nhau theo tiêu chí nào: Id, Name, Price hay kết hợp nhiều property.

10. Hai object cùng dữ liệu vẫn có thể khác nhau

```
Product p1 =  
    new Product("P01", "Mouse", 300);  
  
Product p2 =  
    new Product("P01", "Mouse", 300);
```

Hai lần new tạo ra hai object khác nhau. Nếu Product chưa định nghĩa equality riêng, Distinct() có thể giữ cả p1 và p2 dù dữ liệu property giống nhau.

11. Cùng một object xuất hiện nhiều lần

```
Product product =  
    new Product("P01", "Mouse", 300);  
  
List<Product> products =  
    new List<Product>  
    {  
        product,  
        product,  
        product  
    };
```

Các phần tử trên cùng tham chiếu tới một object. Distinct() sẽ nhận diện chúng là cùng một phần tử và chỉ giữ một.

12. Cách học phù hợp ở bài này

Khi muốn loại trùng theo một property, dùng Select(property).Distinct() là cách rõ và dễ hiểu.

```
products.Select(p => p.Id).Distinct();  
products.Select(p => p.Name).Distinct();  
products.Select(p => p.Price).Distinct();
```

Các cách nâng cao như custom equality, IEquatable<T>, EqualityComparer, DistinctBy() sẽ học/đụng lại khi cần; chưa cần đi sâu ở đây.

PHẦN C - KẾT HỢP DISTINCT VỚI LINQ KHÁC

13. Distinct() + OrderBy()

```
List<decimal> prices =  
    products  
        .Select(p => p.Price)  
        .Distinct()  
        .OrderBy(price => price)  
        .ToList();  
  
Select  
→ lấy Price  
  
Distinct  
→ loại Price trùng  
  
OrderBy  
→ sắp Price tăng dần  
  
ToList  
→ List<decimal>
```

14. Distinct() không phải OrderBy()

```
Distinct  
→ loại trùng  
  
OrderBy  
→ sắp xếp
```

Nếu nghiệp vụ cần thứ tự rõ ràng, hãy gọi OrderBy()/OrderByDescending() riêng.

15. Where + Select + Distinct

```
List<decimal> prices =
    products
        .Where(
            product => product.Price >= 300)
        .Select(
            product => product.Price)
        .Distinct()
        .ToList();

List<Product>
    ↓ Where
IEnumerable<Product>
    ↓ Select
IEnumerable<decimal>
    ↓ Distinct
IEnumerable<decimal>
    ↓ ToList
List<decimal>
```

16. Where + Select + Distinct + OrderBy + ToList

```
products
    .Where(...)
    .Select(...)
    .Distinct()
    .OrderBy(...)
    .ToList();

List<T>
    ↓ Where
IEnumerable<T>
    ↓ Select
IEnumerable<TAnother>
    ↓ Distinct
IEnumerable<TAnother>
    ↓ OrderBy
IOrderedEnumerable<TAnother>
    ↓ ToList
List<TAnother>
```

17. Collection rỗng

Distinct() trên collection rỗng không throw exception. Kết quả là sequence rỗng.

```
emptyCollection.Distinct()
→ empty IEnumerable<T>
```

18. Có thể foreach kết quả Distinct()

Vì Distinct() trả về nhiều phần tử dạng IEnumerable<T>, ta có thể foreach để hiển thị toàn bộ kết quả.

```
foreach (decimal price in uniquePrices)
{
    Console.WriteLine(price);
}
```

PHẦN D - CÂU HỎI LÝ THUYẾT

1. Distinct() dùng để làm gì?
2. Distinct() trả về một phần tử hay một collection?
3. Distinct() có sửa collection gốc không?
4. Với products.Select(p => p.Price).Distinct(), hãy mô tả kiểu dữ liệu qua từng bước.
5. Vì sao Distinct() hoạt động đơn giản với int, decimal, string?
6. Vì sao hai Product có cùng Id, Name, Price vẫn có thể không bị loại bởi Distinct()?
7. Select(p => p.Price).Distinct() khác products.Distinct() thế nào?
8. Distinct() có nhiệm vụ sắp xếp dữ liệu không?

9. Nếu collection rỗng thì Distinct() xảy ra chuyện gì?

10. Với Where -> Select -> Distinct -> OrderBy -> ToList, hãy mô tả chức năng và kiểu dữ liệu qua từng bước.

PHẦN E - BÀI TẬP 10.8

```
List<Product> products =  
    new List<Product>  
    {  
        new Product("P01", "Mouse", 300),  
        new Product("P02", "Keyboard", 700),  
        new Product("P03", "Monitor", 2500),  
        new Product("P04", "Laptop", 15000),  
        new Product("P05", "USB", 150),  
        new Product("P06", "Cable", 300),  
        new Product("P07", "Adapter", 700),  
        new Product("P08", "Mouse", 1200)  
    };
```

- Dùng Select() + Distinct() lấy các mức Price không trùng.
- Lấy các Name không trùng.
- Lấy các Price không trùng rồi sắp tăng dần.
- Lọc Product có Price >= 300, sau đó lấy các mức Price không trùng.
- Chuyển kết quả câu 1 thành List<decimal>.
- Dùng foreach hiển thị kết quả.
- Kiểm tra products gốc không bị thay đổi.
- Tạo List<int> = {1,1,2,2,3,3,3} rồi dùng Distinct().
- Tạo List<string> có "Mouse" và "mouse" rồi quan sát Distinct().
- Tạo hai Product khác nhau nhưng cùng dữ liệu, cho vào List rồi chạy Distinct().

PHẦN F - ĐÁP ÁN CODE

```
public class Program  
{  
    public static void Main(string[] args)  
    {  
        List<Product> products =  
            new List<Product>  
            {  
                new Product("P01", "Mouse", 300),  
                new Product("P02", "Keyboard", 700),  
                new Product("P03", "Monitor", 2500),  
                new Product("P04", "Laptop", 15000),  
                new Product("P05", "USB", 150),  
                new Product("P06", "Cable", 300),  
                new Product("P07", "Adapter", 700),  
                new Product("P08", "Mouse", 1200)  
            };  
  
        var uniquePrices =  
            products  
                .Select(p => p.Price)  
                .Distinct();  
  
        var uniqueNames =  
            products  
                .Select(p => p.Name)  
                .Distinct();  
  
        var orderedUniquePrices =  
            products  
                .Select(p => p.Price)  
                .Distinct()  
                .OrderBy(price => price);  
  
        var filteredUniquePrices =
```

```

        products
            .Where(p => p.Price >= 300)
            .Select(p => p.Price)
            .Distinct();

List<decimal> uniquePriceList =
    uniquePrices.ToList();

foreach (decimal price in uniquePrices)
{
    Console.WriteLine(price);
}

List<int> numbers =
    new List<int>
    {
        1, 1, 2, 2, 3, 3, 3
    };

var uniqueNumbers =
    numbers.Distinct();

List<string> names =
    new List<string>
    {
        "Mouse",
        "mouse",
        "Keyboard",
        "Mouse"
    };

var uniqueStringNames =
    names.Distinct();

Product p1 =
    new Product("P01", "Mouse", 300);

Product p2 =
    new Product("P01", "Mouse", 300);

List<Product> duplicateProducts =
    new List<Product>
    {
        p1,
        p2
    };

var distinctProducts =
    duplicateProducts.Distinct();

foreach (Product product in distinctProducts)
{
    Console.WriteLine(product);
}
}

```

PHẦN G - ĐÁP ÁN LÝ THUYẾT

1. Distinct() dùng để làm gì?

Loại các phần tử trùng theo quy tắc equality và giữ lại một đại diện cho mỗi giá trị khác nhau.

2. Trả một phần tử hay collection?

Trả một sequence/collection, thường hiểu là IEnumerable<T>.

3. Có sửa collection gốc không?

Không. Collection gốc giữ nguyên.

4. Select Price rồi Distinct

List<Product> -> IEnumerable<decimal> sau Select -> IEnumerable<decimal> sau Distinct.

5. Vì sao dễ với int/decimal/string?

.NET đã có quy tắc equality phù hợp cho các kiểu này.

6. Vì sao hai Product cùng dữ liệu vẫn có thể còn cả hai?

Hai lần new tạo hai object khác nhau; nếu Product chưa định nghĩa equality riêng thì Distinct() không tự biết rằng chúng phải được xem là trùng theo Id/Name/Price.

7. Select Price + Distinct khác products.Distinct()

Cách đầu loại trùng theo Price; cách sau loại trùng theo Product object.

8. Distinct() có sắp xếp không?

Không. Distinct() chỉ loại trùng.

9. Collection rỗng

Không throw; trả về sequence rỗng.

10. Chuỗi LINQ

List<T> -> IEnumerable<T> -> IEnumerable<TAnother> -> IEnumerable<TAnother> -> IEnumerable<TAnother> -> List<TAnother>.

PHẦN H - ĐÁNH GIÁ BÀI LÀM VÀ LƯU Ý

- Bài làm ban đầu đạt khoảng 8.6/10; sau phần củng cố câu 8-10, Phần 10.8 được xem là ĐẠT.
- Select + Distinct Price: đúng.
- Select + Distinct Name: đúng.
- Distinct + OrderBy: đúng.
- Where + Select + Distinct: đúng.
- ToList(): đúng.
- Câu 8: List<int> {1,1,2,2,3,3,3} -> Distinct() cho 1,2,3.
- Câu 9: Distinct() mặc định phân biệt "Mouse" và "mouse".
- Câu 10: hai Product được new riêng nhưng cùng dữ liệu có thể vẫn được giữ cả hai nếu chưa định nghĩa equality riêng.
- Lỗi nhỏ về kiểu: sau OrderBy() là IEnumerable<TAnother>, không phải chỉ IEnumerable<TAnother> khi mô tả chính xác kiểu sắp xếp.

1. Điểm quan trọng nhất

Distinct()
→ không chỉ là "xóa trùng"

Cần hỏi:
"Trùng theo cái gì?"

Với value/property đơn giản, Select(property).Distinct() rất rõ ràng. Với object, equality phải được định nghĩa theo quy tắc phù hợp.

2. Cầu nối sang GroupBy()

Distinct()
→ chỉ giữ các giá trị khác nhau

GroupBy()
→ dùng một giá trị làm khóa
và gom các phần tử tương ứng vào từng nhóm

Kết luận: Phần 10.8 - Distinct() đã ĐẠT. Bài tiếp theo là 10.9 - GroupBy().